OTEL-99: Best Practice Summary

> **Series:** OTEL — OpenTelemetry Integration | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/26/2026

Introduction

This notebook consolidates every actionable best practice from the OTEL series (notebooks 01-08) into definitive, reference-ready tables. Each practice specifies the exact setting or value to use, a priority level, and its category. Use this as a checklist when planning, deploying, or auditing OpenTelemetry integration with Dynatrace.

Table of Contents

---

1. Collector Configuration

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always use the `batch` processor	`processors.batch.timeout: 10s`, `send_batch_size: 1000`, `send_batch_max_size: 1500`	Critical	OTEL-02
2	Always use the `memory_limiter` processor	`processors.memory_limiter.check_interval: 1s`, `limit_mib:` 80% of container memory, `spike_limit_mib:` 25% of limit	Critical	OTEL-02
3	Place `memory_limiter` first in every pipeline	`processors: [memory_limiter, resource, batch]`	Critical	OTEL-02, OTEL-08
4	Pin Collector image to a specific version	`image: otel/opentelemetry-collector-contrib:0.120.0` (never `latest`)	Critical	OTEL-03, OTEL-07
5	Validate config before deploy	Run `otelcol-contrib validate --config=config.yaml`	Critical	OTEL-08

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Use the Contrib or Dynatrace distribution	`otel/opentelemetry-collector-contrib` or `ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-collector`	Recommended	OTEL-02, OTEL-07
7	Use `GOMEMLIMIT` instead of deprecated `memory_ballast`	Set `GOMEMLIMIT` env var to ~80% of container memory limit	Critical	OTEL-02
8	Use environment variables for all secrets	`${DT_API_TOKEN}`, `${DT_ENV_ID}` in YAML config	Critical	OTEL-02
9	Add a default `service.name` via resource processor	`processors.resource.attributes: [{key: service.name, value: "unknown-service", action: insert}]`	Recommended	OTEL-07, OTEL-08
10	Enable retry on failure for exporters	`retry_on_failure.enabled: true`, `initial_interval: 5s`, `max_interval: 30s`, `max_elapsed_time: 300s`	Recommended	OTEL-03, OTEL-07

2. Deployment and Sizing

#	Best Practice	Recommended Setting/Value	Priority	Source
11	Use DaemonSet (agent mode) for per-node collection	`kind: DaemonSet` in K8s manifest	Recommended	OTEL-03
12	Use Deployment (gateway mode) for centralized processing	`kind: Deployment` with `replicas: 2+`	Recommended	OTEL-03
13	Size gateway resources by throughput	<1k spans/s: 100m/256Mi; <10k: 500m/1Gi; <100k: 2/4Gi; >100k: 4+/8Gi+	Critical	OTEL-03
14	Set `memory_limiter.limit_mib` to 80% of container memory limit	Container limit 1Gi -> `limit_mib: 800`	Critical	OTEL-03
15	Deploy gateway with minimum 2 replicas for HA	`spec.replicas: 2`	Recommended	OTEL-03
16	Add PodDisruptionBudget for gateway	`minAvailable: 1`	Recommended	OTEL-03
17	Use HorizontalPodAutoscaler at 70% CPU target	`averageUtilization: 70`, `minReplicas: 2`, `maxReplicas: 10`	Recommended	OTEL-03
18	Enable persistent queue for gateway exporters	`sending_queue.enabled: true`, `queue_size: 1000`, `storage: file_storage`	Recommended	OTEL-03
19	Install via Helm for K8s deployments	`helm install otel-collector open-telemetry/opentelemetry-collector --set mode=deployment`	Recommended	OTEL-03
20	Set `TZ=UTC` on Collector containers	`env: [{name: TZ, value: UTC}]`	Recommended	OTEL-08

3. Security

#	Best Practice	Recommended Setting/Value	Priority	Source
21	Store API tokens in Kubernetes Secrets	`kind: Secret`, reference via `secretKeyRef`	Critical	OTEL-03
22	Enable TLS on receiver endpoints	`tls.cert_file`, `tls.key_file` on `otlp.protocols.grpc` and `.http`	Critical	OTEL-03
23	Enable mTLS for internal traffic	`tls.client_ca_file` on receiver	Optional	OTEL-03
24	Apply NetworkPolicy to restrict Collector ingress/egress	Allow ingress 4317/4318 from app namespaces; egress 443 to Dynatrace only	Recommended	OTEL-03
25	Strip sensitive attributes in the processor pipeline	`processors.attributes.actions: [{key: api.key, action: delete}]`	Critical	OTEL-02
26	Never embed tokens in code or config files	Always use env vars or Secret references	Critical	OTEL-07
27	Never include PII in span or metric attributes	Exclude user emails, SSNs, credit card numbers	Critical	OTEL-04

4. Dynatrace OTLP Integration

#	Best Practice	Recommended Setting/Value	Priority	Source
28	Use OTLP/HTTP for direct Dynatrace ingest	`exporters.otlphttp.endpoint: https://{env-id}.live.dynatrace.com/api/v2/otlp`	Critical	OTEL-01, OTEL-07
29	Route gRPC through a Collector (gRPC is not supported for direct DT ingest)	Collector with `otlp` gRPC receiver + `otlphttp` exporter to Dynatrace	Critical	OTEL-01, OTEL-07
30	Use `Api-Token` prefix in Authorization header	`headers: {Authorization: "Api-Token <token>"}`	Critical	OTEL-07
31	Create a dedicated token with minimum required scopes	`openTelemetryTrace.ingest` + `metrics.ingest` + `logs.ingest`	Critical	OTEL-01, OTEL-07
32	Use Dynatrace Collector distribution for production	`ghcr.io/dynatrace/dynatrace-otel-collector/dynatrace-otel-collector`	Recommended	OTEL-07
33	Use ActiveGate endpoint for on-prem or network-restricted envs	`https://{activegate-host}:9999/e/{env-id}/api/v2/otlp`	Recommended	OTEL-07
34	Use Dynatrace Operator OTLP auto-config for K8s (v1.8+)	Annotation `otlp-exporter-configuration.dynatrace.com/inject` for per-pod control	Recommended	OTEL-07
35	After enabling advanced OTLP metric dimensions, stop relying on auto-enriched `dt.entity.service`	Filter using `service.name` instead of `dt.entity.service` in SLOs, alerts, dashboards	Critical	OTEL-07

5. Resource Attributes and Entity Mapping

#	Best Practice	Recommended Setting/Value	Priority	Source
36	Always set `service.name` resource attribute	`resource.create({"service.name": "checkout-api"})`	Critical	OTEL-07, OTEL-08
37	Set `service.version` for release tracking	`"service.version": "1.2.3"`	Recommended	OTEL-07
38	Set `service.namespace` for service grouping	`"service.namespace": "ecommerce"`	Recommended	OTEL-07
39	Set `deployment.environment.name` for environment identification	`"deployment.environment.name": "production"` (use stable name, not legacy `deployment.environment`)	Recommended	OTEL-01, OTEL-07
40	Set K8s resource attributes for entity linking	`k8s.namespace.name`, `k8s.pod.name`, `k8s.deployment.name`	Recommended	OTEL-01, OTEL-07
41	Use `resourcedetection` processor for cloud/K8s metadata	`processors.resourcedetection` with env, gcp, aws, azure, or k8s detectors	Recommended	OTEL-02

6. Trace Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
42	Start with auto-instrumentation, add manual spans for business logic	Auto-instrument frameworks; add custom spans with `tracer.start_as_current_span()`	Recommended	OTEL-04
43	Use low-cardinality span names	`HTTP GET /users/{id}` not `HTTP GET /users/12345`	Critical	OTEL-04
44	Use descriptive, operation-based span names	`process_order`, `db.query` not `order_abc123`, `SELECT * FROM...`	Critical	OTEL-04
45	Always record errors on spans	`span.record_exception(e)` + `span.set_status(StatusCode.ERROR, str(e))`	Critical	OTEL-04
46	Add business context as span attributes	`span.set_attribute("order.id", order_id)`	Recommended	OTEL-04
47	Use span events for key milestones	`span.add_event("payment_completed", {"transaction_id": txn_id})`	Optional	OTEL-04
48	Use `BatchSpanProcessor` (never `SimpleSpanProcessor` in production)	`provider.add_span_processor(BatchSpanProcessor(exporter))`	Critical	OTEL-04
49	Use W3C Trace Context propagation (required for OneAgent interop)	`W3CTraceparentPropagator` + `W3CTracestatePropagator`	Critical	OTEL-04, OTEL-07

#	Best Practice	Recommended Setting/Value	Priority	Source
50	Propagate context across async boundaries	Attach/detach context explicitly in async code	Critical	OTEL-04, OTEL-08
51	Propagate context in messaging (Kafka, RabbitMQ)	Inject context into message headers on produce; extract on consume	Recommended	OTEL-04
52	Use `TraceIdRatioBased` sampler to control trace volume	`TraceIdRatioBased(0.1)` for 10% sampling in high-throughput envs	Recommended	OTEL-08

7. Metrics Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
53	Choose the correct instrument type	Counter for monotonic; Histogram for durations/sizes; UpDownCounter for gauges; ObservableGauge for current state	Critical	OTEL-05
54	Use hierarchical metric naming	`<domain>.<component>.<metric>` e.g. `http.server.request.duration`	Recommended	OTEL-05
55	Include units in metric names or unit field	`unit="ms"` for duration, `unit="By"` for bytes	Recommended	OTEL-05
56	Keep attribute cardinality low	Use route templates (`/users/{id}`), status classes (`2xx`), never user IDs or timestamps	Critical	OTEL-05
57	Configure explicit histogram bucket boundaries	`boundaries: [5, 10, 25, 50, 100, 250, 500, 1000, 2500, 5000, 10000]` for response times	Recommended	OTEL-05
58	Use Views to drop noisy internal metrics	`View(instrument_name="internal.debug.*", aggregation=DropAggregation())`	Optional	OTEL-05
59	Use `ObservableGauge` / `ObservableCounter` for system stats	Avoid polling in the request hot path	Recommended	OTEL-05
60	Set appropriate export interval	`PeriodicExportingMetricReader(exporter, export_interval_millis=60000)` (60s default)	Recommended	OTEL-05
61	Measure RED metrics for services	Rate (Counter), Errors (Counter), Duration (Histogram)	Critical	OTEL-05
62	Measure USE metrics for resources	Utilization, Saturation, Errors	Recommended	OTEL-05

8. Logs Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Source
63	Use the Log Bridge pattern (do not replace your logging library)	Attach `LoggingHandler` to standard Python logger; use Log4j/Logback appenders for Java	Critical	OTEL-06

#	Best Practice	Recommended Setting/Value	Priority	Source
64	Enable automatic log-trace correlation	Use OTel tracing + log bridge together; `trace_id` and `span_id` are injected automatically	Critical	OTEL-06
65	Use structured JSON logging	Output logs as JSON objects with `timestamp`, `level`, `message`, and structured fields	Recommended	OTEL-06
66	Filter DEBUG/TRACE logs at the Collector, not in the app	`processors.filter.logs.log_record: ['severity_number < 9']`	Recommended	OTEL-06
67	Use `BatchLogRecordProcessor` for log export	Never use synchronous log export in production	Critical	OTEL-06
68	Use filelog receiver for container/file-based log collection	`receivers.filelog.include: ["/var/log/containers/*.log"]` with JSON and severity parsers	Recommended	OTEL-06
69	Set log severity appropriately	ERROR: unexpected failures; WARN: recoverable issues; INFO: business events; DEBUG: dev only	Recommended	OTEL-06
70	Never log PII or secrets	Exclude passwords, tokens, personal data from log bodies and attributes	Critical	OTEL-06

9. Semantic Conventions

#	Best Practice	Recommended Setting/Value	Priority	Source
71	Migrate to stable HTTP semantic conventions	Use `http.request.method` (not `http.method`), `http.response.status_code` (not `http.status_code`), `url.full` (not `http.url`)	Recommended	OTEL-01
72	Migrate to stable DB semantic conventions	Use `db.namespace` (not `db.name`), `db.query.text` (not `db.statement`), `db.operation.name` (not `db.operation`)	Recommended	OTEL-01
73	Use `OTEL_SEMCONV_STABILITY_OPT_IN` to control migration	`http` for stable-only, `http/dup` for dual-emit during migration; same pattern for `database`	Recommended	OTEL-01
74	Use stable resource attribute `deployment.environment.name`	Not the legacy `deployment.environment`	Recommended	OTEL-01
75	Follow semantic conventions for all standard attributes	Use `service.name`, `service.version`, `k8s.namespace.name`, `k8s.pod.name`, etc.	Critical	OTEL-01

10. Performance and Reliability

#	Best Practice	Recommended Setting/Value	Priority	Source
76	Use `spanmetrics` connector to derive metrics from traces	`connectors.spanmetrics` with histogram buckets `[100ms, 500ms, 1s, 5s]`	Optional	OTEL-02
77	Filter health-check spans at the Collector	`processors.filter.traces.span: ['attributes["http.url"] == "/health"']`	Recommended	OTEL-02
78	Use fan-out to send data to multiple backends	Define multiple named exporters under `exporters` and list all in pipeline	Optional	OTEL-02

#	Best Practice	Recommended Setting/Value	Priority	Source
79	Increase `sending_queue.num_consumers` for high throughput	`num_consumers: 10`	Recommended	OTEL-08
80	Do not over-instrument	Only create spans for meaningful operations; avoid spans on every function call	Recommended	OTEL-04
81	Limit attribute value sizes	Avoid multi-KB strings in span attributes or log bodies	Recommended	OTEL-04
82	Configure proxy for network-restricted environments	`exporters.otlphttp.proxy_url: http://proxy.example.com:8080` or `HTTPS_PROXY` env var	Optional	OTEL-08

11. Troubleshooting and Observability

#	Best Practice	Recommended Setting/Value	Priority	Source
83	Enable `health_check` extension	`extensions.health_check.endpoint: 0.0.0.0:13133`	Critical	OTEL-02, OTEL-08
84	Enable `zpages` extension in non-production	`extensions.zpages.endpoint: 0.0.0.0:55679`	Recommended	OTEL-02, OTEL-08
85	Use `debug` exporter during development	`exporters.debug.verbosity: detailed` alongside production exporter	Recommended	OTEL-02, OTEL-08
86	Set Collector log level to `info` in production, `debug` for troubleshooting	`service.telemetry.logs.level: info` (default), switch to `debug` when investigating	Recommended	OTEL-08
87	Verify tracer provider is not NoOp	In Python: `print(trace.get_tracer_provider())` must not show `NoOpTracerProvider`	Critical	OTEL-08
88	Test connectivity with curl before deploying	`curl -v https://{env}.live.dynatrace.com/api/v2/otlp/v1/traces -X POST -H "Authorization: Api-Token <token>"`	Recommended	OTEL-08
89	Troubleshoot systematically: SDK -> Collector -> Backend	Follow the data path; isolate which component is failing	Recommended	OTEL-08
90	Enable `pprof` extension for Go heap profiling	`extensions.pprof.endpoint: 0.0.0.0:1777`	Optional	OTEL-08

Summary

This notebook contains **90 best practices** extracted from the OTEL series, organized into 11 categories:

Category	Count	Critical	Recommended	Optional
Collector Configuration	10	6	4	0
Deployment and Sizing	10	2	8	0
Security	7	5	1	1
Dynatrace OTLP Integration	8	5	3	0
Resource Attributes and Entity Mapping	6	1	5	0
Trace Instrumentation	11	6	4	1
Metrics Instrumentation	10	3	6	1
Logs Instrumentation	8	4	3	1
Semantic Conventions	5	1	4	0
Performance and Reliability	7	0	4	3
Troubleshooting and Observability	8	2	5	1
Total	**90**	**35**	**47**	**8**

Priority guide:
- **Critical** -- Implement before going to production. Skipping causes data loss, security exposure, or broken entity mapping.
- **Recommended** -- Implement for production-grade deployments. Improves reliability, performance, and maintainability.
- **Optional** -- Implement when the specific use case applies. Adds value in targeted scenarios.

References

- Dynatrace OpenTelemetry Integration
- Dynatrace Collector Distribution
- OTLP API Endpoints
- Configure OTLP Metrics
- OpenTelemetry Official Docs
- OTel Collector Configuration
- Semantic Conventions
